

MTSecurity: Privacy-Preserving Malicious Traffic Classification Using Graph Neural Network and Transformer

Jin Yang^{ID}, Xinyun Jiang^{ID}, Yulin Lei^{ID}, Weiheng Liang^{ID}, Zicheng Ma^{ID}, and Siyu Li^{ID}

Abstract—Encrypting network traffic is an effective means of safeguarding user privacy and sensitive information. However, it also introduces potential vulnerabilities that can be exploited by network attackers, posing significant security risks to the Internet. In response to the challenge of low accuracy in existing methods for classifying encrypted malicious traffic, we propose a novel approach named MTSecurity, which leverages Transformer and Graph Neural Network technologies. This method automatically extracts raw byte features and graph-based traffic interaction features from encrypted malicious flows, combining them to substantially enhance the classification accuracy of encrypted malicious traffic. Furthermore, we introduce a graph structure called the Malicious Traffic Interaction Graph (MTIG) for representing encrypted malicious traffic. MTIG is based on the client-server interaction process and incorporates multi-dimensional traffic features. Experimental results demonstrate that the proposed MTSecurity model consistently performs well across different datasets, surpassing state-of-the-art methods. It achieves an accuracy of 0.9946 and an F1 score of 0.9940 on the MCFP dataset, and an accuracy of 0.9948 with an F1 score of 0.9934 on the USTC-TFC dataset.

Index Terms—Network intrusion detection, encrypted malicious traffic classification, deep learning, graph neural networks, transformer.

I. INTRODUCTION

AS INTERNET users increasingly care about privacy protection and information security, more and more applications employ encryption mechanisms to secure data transmission. Browser vendors such as Google and Mozilla have begun to force the use of secure transmission protocols such as Hypertext Transfer Protocol Secure (HTTPS), and browsers issue warnings that communications are initiated without the use of such protocols. Moreover, Google has announced plans to realize 100% encryption in all of its

products and services [1]. Although applying encryption in data transmission can effectively preserve data confidentiality and integrity, this measure can also bring new security risks and threats. By encrypted traffic, malicious software (such as phishing software, ransomware, trojans, and others) can evade the security checks of firewalls and traditional intrusion detection systems. Furthermore, network attackers can use encryption techniques to conceal cyberattacks and conduct data thefts undetected. In its annual hotspot threat report released in 2019, Cisco pointed out that 63% of all threat incidents discovered by Cisco used encrypted traffic [2]. According to the encryption attack report released by the cloud security vendor Zscaler, there were more than 20 billion HTTPS-based threats in 2021, an increase of 314% over 2020. Malware using encrypted channels has increased by 212%, while ransomware is becoming the tool of choice for more and more cyber-attackers seeking profit [3].

Unlike its unencrypted counterpart, encrypted malicious traffic detection is difficult. After the payload has been encrypted, its upper layer becomes invisible, which renders traditional security systems based on firewalls and deep packet inspection ineffective. However, decryption-based detection methods are costly and could be considered as violations of user privacy. Therefore, detection and classification of malicious traffic under encryption is important and significant. There have been attempts to utilize statistical characterization of network flows, such as IP address, port, server name and cipher in encrypted malicious traffic classification [4], [5], [6], [7]. Statistical feature-based methods have low accuracy and require human involvement for feature sampling. Recent studies have proposed a number of methods based on raw traffic data [8], [9], [10], [11], [12]. These methods perform well by applying deep learning models such as convolutional neural network (CNN). However, most schemes mainly rely on packet payload, which have the potential to violate user privacy. Unlike regular encrypted traffic, malicious traffic such as ransomware and data theft may have access to the user's secret files, which are encrypted but still have the potential to be decrypted and utilized. Users may not want this information to be used by anyone when performing traffic analysis.

To address the above problems, in this paper we propose MTSecurity, an encrypted malicious traffic classification method using Graph Neural Networks (GNNs) and Transformer. The proposed model can identify malicious

Manuscript received 30 September 2023; revised 22 January 2024; accepted 26 March 2024. Date of publication 1 April 2024; date of current version 12 July 2024. This work is supported by the National Natural Science Foundation of China under Grants No. 61872254 and No. 62162057; the Key Lab of Information Network Security of Ministry of Public Security under Grant No. C20606, and the Sichuan Science and Technology Program under Grant No. 2021JDR0004. The associate editor coordinating the review of this article and approving it for publication was F. Valenza. (Jin Yang and Xinyun Jiang contributed equally to this work.) (Corresponding author: Yulin Lei.)

The authors are with the School of Cyber Science and Engineering, Sichuan University, Chengdu 610065, China (e-mail: jinyangscu@163.com; jiangxinyun@stu.scu.edu.cn; yulinleiscu@163.com; liangweiheng@stu.scu.edu.cn; mazicheng@stu.scu.edu.cn; lisiyu_real@stu.scu.edu.cn).

Digital Object Identifier 10.1109/TNSM.2024.3383851

traffic classes among a large number of benign traffic. We use packet bytes without user payload for privacy-preserving traffic analysis. In addition, we construct a malicious traffic interaction graph (MTIG) to represent the interaction process of client-server of malicious traffic flow, which is inspired by Shen et al. [6]. The MTIG is constructed without involving sensitive information of users, and can reveal network behavior patterns and multi-dimensional traffic characteristics of malicious traffic. The proposed method further enhances the ability to represent encrypted malicious traffic by fusing the raw data features of the flow with the graph features of the flow, which results in significantly improved classification accuracies.

The main contributions of this paper are threefold:

(1) We propose MTSecurity, which can detect encrypted malicious traffic by fusing the byte features of traffic with the graph-based traffic interaction features. MTSecurity can describe the behavior of malicious traffic more comprehensively than any individual analysis. In addition, MTSecurity does not use traffic payload information and therefore does not compromise user privacy.

(2) We construct a Malicious Traffic Interaction Graph (MTIG) to represent malicious traffic at the network interaction level. Using MTIG, the packet burst information, traffic intrinsic characteristics and network interaction information of malicious flows are preserved, which can enhance flow representation and serve as discriminative features for classifiers learning.

(3) We design a Graph Neural Network (GNN) module to automatically extract graph representation features. It can map the input MTIG to the high-dimensional embedding space to form different representations to distinguish different graph structures, avoiding the manual creation of features, and therefore realizing efficient and accurate graph feature extraction. Meanwhile, we validate the performance of the method on publicly available real malicious traffic datasets. The experimental results demonstrate the effectiveness of MTSecurity, which achieves a detection accuracy of 0.9949 on the MCFP dataset [30] and 0.9946 on the USTC-TFC dataset [31] using enhanced traffic characterization, outperforming the state-of-the-art methods.

The remainder of this paper is organized as follows. The related work is summarized in Section II and the preliminaries required in the understanding of our proposed scheme is presented in Section III. Then, the framework of the algorithm and each module are introduced in detail in Section IV. The experimental setup is described in Section V. The performance of MTSecurity is evaluated in Section VI and compared with other state-of-the-art methods. Finally, the threats to validity are discussed in Section VII and the paper is concluded in Section VIII.

II. RELATED WORK

Encrypted malicious traffic detection and classification has been a research hotspot in recent years. In this section, we reviewed and summarized those methods that are closely associated to our work, which can be sorted roughly into three categories as below.

A. Methods Based on Statistical Features

Most existing encrypted traffic classification studies are aimed at statistical features of network traffic, such as packet length, packet time, etc. For example, Shekhawat et al. [4] exploited the statistical information such as IP address, port, server name and cipher for encrypted malicious traffic analysis. Taylor et al. [5] utilized statistical features of packet length and Shen et al. [6] utilized statistical features of accumulated packet length to train the random forest classifier for encrypted traffic classification. Wang et al. [7] summarized the general features of encrypted traffic and obtained 113 protocol-independent numerical features to identify malicious encrypted traffic. Barut et al. [27] used pre-extracted features and employed deep learning models. Several studies have used SSL/TLS flags in encrypted traffic and applied Markov models to categorize different smartphone applications [33], [34].

The advantages of utilizing statistical feature-based methods lie in their quick training and efficient feature matching capabilities. However, these methods exhibit drawbacks such as low accuracy and the necessity for human involvement in feature selection. The selection of features has a direct impact on the model's performance. Additionally, over time and with alterations in the network environment, features that were once effective may become obsolete, prompting the need for their re-selection to train a new model.

B. Methods Based on Raw Traffic Data

Encrypted traffic detection methods based on raw traffic have become a popular approach. This is attributed to the fact that directly analyzing raw traffic data eliminates the need for manual feature selection and design. Wang et al. [8] preprocessed the raw traffic into IDX3 files that were fed to convolutional neural network to automatically extract encrypted traffic representation. Shapira and Shavitt [9] transformed flow data into pictures and adopted deep learning techniques to identify different encrypted traffic categories. Lotfollahi et al. [10] proposed Deeppacket to analyze raw traffic payloads by using stacked autoencoder (SAE) and convolution neural network, while Lin et al. [11] also exploited raw traffic data by using convolution neural network and Long-Short Term Memory (LSTM). Last year, Lin et al. [12] introduced ET-BERT, which converted raw traffic into token representations to feed a bi-directional encoder representation transformer (BERT) for encrypted traffic and malicious traffic classification. Barut et al. [19] proposed a deep learning model that used raw traffic as input and processed the traffic to avoid user privacy leakage.

However, although some researchers have considered privacy issues, most of these methods still use the payload of the packets during the preprocessing phase thus potentially violating the user's privacy. Moreover, these methods focus only on the byte data of the captured packets, ignoring the network interaction process, which contains multi-dimensional traffic characteristics.

C. Methods Based on Graph Representations

Graph-based approaches have gradually attracted the attention of researchers because they can represent the

TABLE I
SUMMARY OF RELEVANT LITERATURE

Ref.	Dataset	Input	Privacy Preserving	Method
[4]	CTU-13, MCFP	Statistic Feature	Yes	Machine Learning
[5]	Private	Statistic Feature	Yes	Machine Learning
[27]	NetML, CICIDS2017, VPN2016	Statistic Feature	Yes	Deep Learning
[7]	UNSW-NS, CICIDS2017 etc.	Statistic Feature	Yes	Machine Learning
[19]	NetML, CICIDS2017	Raw Data	Yes	Deep Learning
[9]	ISCX-VPN, ISCX-Tor	Raw Data	No	Deep Learning
[10]	ISCX VPN-non VPN	Raw Data	No	Deep Learning
[11]	ISCX-Tor, USTC-TFC	Raw Data	No	Deep Learning
[12]	USTC-TFC, ISCX-Tor etc.	Raw Data	No	Deep Learning
[13]	Private	Graph Feature	Yes	Deep Learning
[15]	CICIDS2017, CIDDos2017	Graph Feature	Yes	Deep Learning

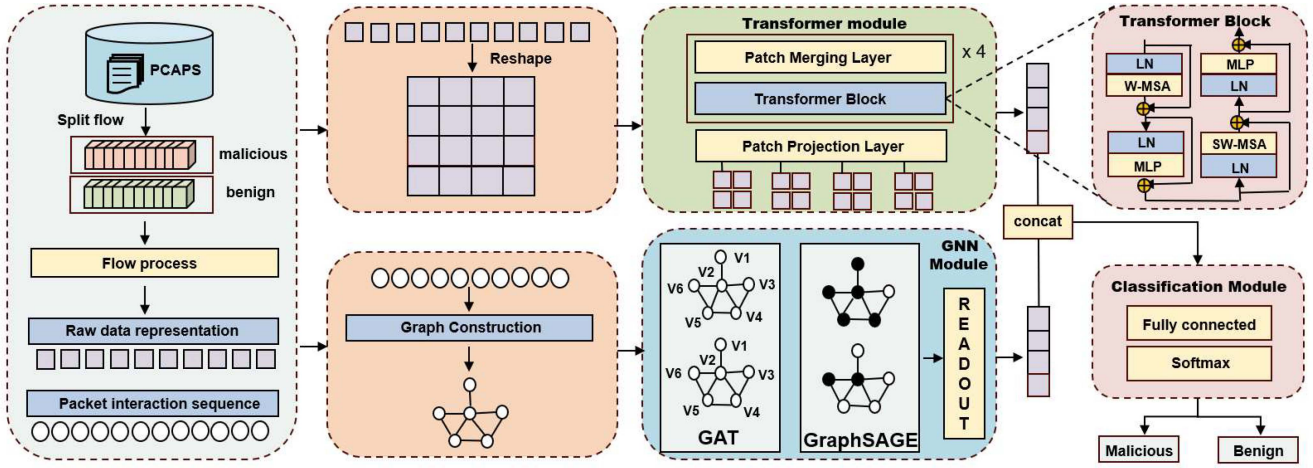


Fig. 1. The overall architecture of MTSecurity model.

difference between normal and abnormal access without considering user payload. Wang et al. [14] constructed a communication graph to achieve accurate detection of botnets by observing the domains of normal and abnormal nodes. Shen et al. [13] constructed a traffic graph structure from the server-client interaction (TIG). Using graph representations, the traffic classification problem can be transformed into a graph classification problem. Li et al. [15] converts malicious traffic into endpoint traffic graphs by analyzing the relationships between packets and between flows in traffic, and learns to classify graph-structured DDoS attack patterns adaptively through graph neural networks.

Graph-based representations do not use packet data and therefore do not reveal the sensitive information of users. But the accuracy of graph-based malicious traffic classification methods is closely related to the pattern of the constructed graph. When the malicious flows are short, different types of malicious traffic will generate similar graph structures thus causing this method to fail. To address this problem, this paper proposes a new graph structure which can enhance the representation of shorter malicious flows by embedding multi-dimensional features into graph nodes, thereby significantly improving the classification accuracy of malicious encrypted traffic.

III. PRELIMINARIES

Malicious traffic classification trying to associate specific abnormal traffic with the corresponding types. In this paper, we try to construct a new method which trains the Transformer and GNN models to extract features at the byte level and network interaction level of network traffic, respectively, and fuses them to accurately distinguish encrypted malicious traffic from benign encrypted traffic. In this section, we will briefly introduce some preliminaries to facilitate the understanding of the proposed approach.

A. Packet Byte Data

Network traffic consists of binary bytes of data. Traffic packets encapsulate user information hierarchically according to the network protocol. Given a network flow with n packets, we denote it as:

$$flow = [P_1, P_2, \dots, P_n] \quad (1)$$

For any of these TLS protocol-based packets P_i , assume that the total number of bytes in the packet is l . Then P_i can be denoted as:

$$P_i = [b_1, b_2, \dots, b_l] \quad (2)$$

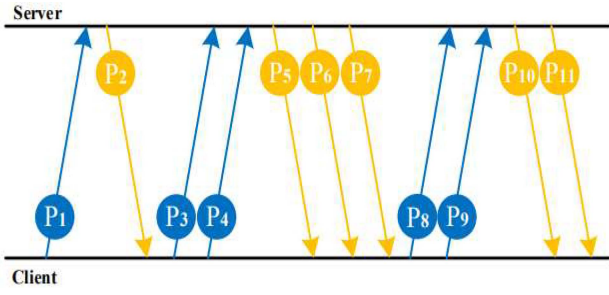


Fig. 2. The client-server interaction of malicious traffic flow on packet-level.

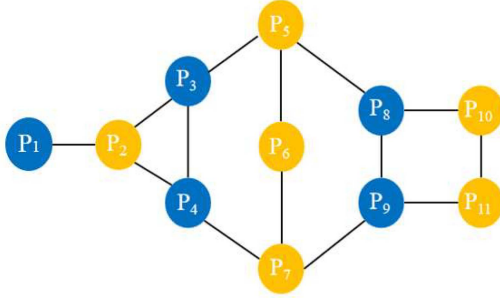


Fig. 3. Constructing Malicious Traffic Interaction Graph (MTIG) based on client-server interaction.

Different malicious flows behave differently in terms of byte content. These differences in byte content are related to specific traffic types. In specific practice, the bytes where the IP address and port are located are removed and the TLS payload portion is discarded to protect user privacy.

B. Network Interactions

When a TCP connection is created between a client and a server, client will usually send some requests to the server for requesting multiple resources. Fig. 2. illustrates the typical client-server interaction process.

Uplink-packet: Consider the packets sent from the client to the server as uplink packets. When a long-lived TCP connection is established by the client for more efficient data transmission, so that multiple continuous uplink packets can be observed. The one-way continuous packets form a burst. The burst [13], [16] is defined as a series of consecutive data packets transmitted in the same direction over a period of time.

Downlink-packet: Consider the packets sent from the server to the client as downlink packets. The downlink packet means that the server starts transmitting the content required by the client.

When a malware client communicates with the malware server, the interaction process can be used to express resulting traffic trace. For example, constructing a malicious traffic interaction graph (the graph construction algorithm is described in Section IV-C).

C. Self-Attention

Self-attention (SA) is popular in Transformers architecture [17]. SA can be computed in parallel, remembering the

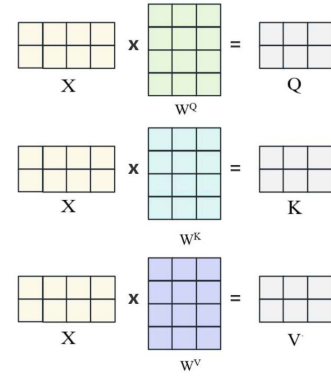


Fig. 4. The arithmetic procedure of Q, K, V.

correlation between inputs, and has become the main choice for deep learning algorithms.

Network traffic is usually continuous, and multiple packets are strongly correlated with each other in terms of content. This relationship needs to be extracted efficiently. The use of SA in malicious traffic classification can compute the correlation between the previous packet and the next packet in terms of byte content, so that the relationship between packet contents can be fully extracted. Before SA, traffic classification usually used models like RNN, LSTM which can process packets chronologically, but these algorithms cannot be parallelized and limit the performance. The formula for SA is as follows:

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (3)$$

where Q, K, V are the query, key and value matrix, respectively, and d is the dimension of query/key. Assuming that the input matrix is X, Q, K, V can be expressed as follows (4)–(6):

$$Q = XW^Q \quad (4)$$

$$K = XW^K \quad (5)$$

$$V = XW^V \quad (6)$$

where W^Q , W^K and W^V are three trainable parameter matrices. The input matrix X is multiplied with W^Q , W^K and W^V to generate Q, K and V, respectively, which is equivalent to undergoing a linear transformation. Instead of using X directly, *Attention* uses these three trainable parameter matrices generated after matrix multiplication to enhance the fitting ability of the model [39]. The arithmetic procedure of Q, K, V is shown in Fig. 4.

Usually, in order to get the results of SA in different subspaces, multiple SAs are used in parallel, and then the results of each SA are concatenated to get the multi-head self-attention (MSA):

$$MSA = Concat(head_1, \dots, head_n)W^O \quad (7)$$

where $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$.

D. Graph Neural Network

Graph neural networks mainly deal with non-Euclidean data (e.g., chemical molecular structures, knowledge graphs,

TABLE II
LIST OF ABBREVIATIONS

Symbol	Description
G	A graph.
V	The set of nodes in a graph.
E	The set of edges in a graph.
v	A node $v \in V$.
$N(v)$	The neighbors of a node v .
d	The dimension of a node feature vector.
b	The dimension of a hidden node feature vector.
c	The dimension of an edge feature vector.
$x_v \in \mathbf{R}^d$	The feature vector of the node v .
$h_v \in \mathbf{R}^b$	The hidden feature vector of node v .
$X_{uv}^e \in \mathbf{R}^c$	The edge feature vector of the edge (u, v) .
k	The index of layer.
$S_{N(v)}$	Random sample of the node v 's neighbors.
h_G	Feature representation of graph G .

malicious traffic interaction graphs, etc.) to make up for the shortcomings of deep learning algorithms such as Transformer, CNN, etc., in extracting features from non-Euclidean data. Inspired by convolutional neural networks, the concept of graph convolution has been rapidly developed. The basic idea of graph convolution is that a node v generates a hidden representation of the node v by aggregating its own information x_v and the information of its neighboring nodes x_u , where $u \in N(v)$. The notations used in this paper are shown in Table II.

A fundamental paradigm in convolutional graph neural networks (e.g., GAT, GraphSAGE) is the message-passing neural network [18]. It treats graph convolution as a kind of message passing process. In this process, messages are passed along edges from one node to another. The message is further propagated by running k -step message passing iterations. The message passing function is defined as:

$$h_v^{(k)} = U_k \left(h_v^{(k-1)}, \sum_{u \in N(v)} M_k \left(h_v^{(k-1)}, h_u^{(k-1)}, X_{vu}^e \right) \right) \quad (8)$$

where $h_v^{(0)} = x_v$, which denotes the feature on node v . X_{uv}^e denotes the features on edge (u, v) . $U_k(\cdot)$ and $M_k(\cdot)$ are functions with learnable parameters. After deriving the hidden representation of each node, $h_v^{(k)}$ can be passed to the output layer to perform node-level prediction tasks or to the readout layer to perform graph-level prediction tasks. The readout function generates a representation of the entire graph based on the node hidden representation, which is defined as:

$$h_G = R \left(h_v^{(k)} | v \in G \right) \quad (9)$$

where $R(\cdot)$ denotes a readout function with learnable parameters.

IV. THE PROPOSED MTSECURITY

In this section, we illustrate the procedure of the proposed malicious traffic classification method named MTSecurity. We first describe the entire framework of MTSecurity. Secondly,

we illustrate the raw traffic characterization, where packet payloads are discarded to protect user privacy. Thirdly, we detail the process of constructing a malicious traffic interaction graph, which is inspired by the server-client interaction process and can represent multidimensional network features. After that, byte feature extraction module and graph feature extraction will be proposed separately. Finally, we propose the classification module.

A. Method Overview

The overall framework of our method is shown in Fig. 1. In the data generation phase, since the capture packet (PCAP) files usually composed of multiple flows, we first split the PCAP into independent bi-directional flows (bi-flows). After that, each bi-flows generates the original traffic representation and malicious flow interaction graph after the raw traffic characterization (Section IV-B) and MTIG construction (Section IV-C), respectively.

In the feature extraction phase, the raw traffic characterization consisting of traffic byte data will be computed by the transformer module to extract traffic byte level features, which are related to specific traffic types. At the same time, our constructed malicious traffic interaction graph will be exported to the GNN module to automatically extract traffic interaction level features.

In the traffic classification phase, the generated traffic byte-level features and network interaction-level features are fused to enhance the traffic characterization and go through a softmax layer for multi-task/multi-class malicious traffic classification.

B. Raw Traffic Characterization

1) *Raw Traffic Parsing and Anonymization*: The serialized bytes of data are obtained by parsing the traffic packets according to the network protocol layers. Since link-layer headers contain only information about the physical connection between devices, we discard Ethernet frame headers containing MAC addresses and link information and start parsing at the IP layer. Although the migration from IPv4 to IPv6 is underway, most network traffic is still IPv4-based, so our approach uses IPv4 traffic. The IP addresses and port numbers in the packets are anonymized to preserve user privacy and maintain the generalizability of the model. More specifically, we removed the bytes data where the source/destination IP addresses and source/destination ports are located. During the shift of malicious traffic to encrypted trend, some non-encrypted malicious traffic still exists and the proposed method needs to be compatible with non-encrypted malicious traffic. Thus, in addition to SSL/TLS flows, non-encrypted flows are also parsed. Specifically, for TLS session flows, only the unencrypted handshake record of TLS flows was parsed while the encrypted transmission data was discarded. For HTTP session flows, only the HTTP headers were parsed. Specifically, it means that the data before the two Carriage Return and Line Feeds in the HTTP header is parsed, and the data after that is discarded. Unlike TLS, HTTP protocols, DNS mainly implements the mapping between IP addresses

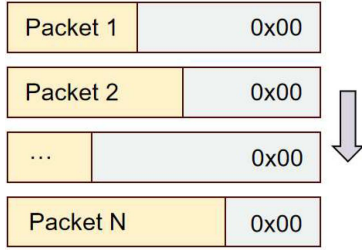


Fig. 5. The vectorization method of a traffic flow.

and domain names and does not directly transfer user data. Referring to [19], we use DNS for all request and response data. In addition, due to the richness of network protocols in real network environments, e.g., File Transfer Protocol, MySQL protocol, etc. To achieve scalability, we parse other reliable transport-based protocols to the header of TCP, while TCP payloads are discarded. And we parse other unreliable transport-based protocols to the header of UDP, while UDP payloads are discarded.

2) *Raw Traffic Consistent*: Since the packet length and the size of individual packets in a network stream are not fixed, the first N packets are chosen to represent the entire data flow to obtain a consistent data representation. If the value of N is assigned reasonably, the first few packets of a data flow can generally contain the basic characteristics of the data flow [20]. Also, the first L bytes of the processed packet are used to represent the packet. At this point, the data packet has been parsed and anonymized into a payload-independent byte sequence. If the length of processed packet is less than the truncated value, we will pad the empty part with zeros. Similarly, if the number of packets in a flow is fewer than N packets, data processing will be applied in the same way. Fig. 5 illustrates a schematic diagram of a packet after length consistency.

After that, splice the byte data of N packets with a length of L in chronological order. And a $1 \times M$ data vector was obtained, where $M = N \times L$. All bytes in the data vector will be converted to integers from 0 to 255. To make it easier for model to train, all data are normalized by dividing by 255. Then the processed one-dimensional raw data vector will be transformed into a two-dimensional matrix as the input of Transformer module. Since the input to our Transformer module is set to x^2 , but M is not guaranteed to be squared. To ensure that valid information is used and avoid more padding with zeros, select $\sqrt{M}$ as an upper limit for the size of the input data (which means $x \leq \lfloor \sqrt{M} \rfloor$), and the excess is discarded. And x usually takes an integer very close to $\sqrt{M}$.

C. MTIG Construction

The MTIG is constructed based on the malicious traffic interaction process. Given a specific network flow, a corresponding MTIG can be generated. Each packet in the flow represents a node in the graph. After determining the graph nodes, edges are added to connect these nodes.

TABLE III
CHARACTERIZATION OF MTIG NODE EMBEDDING

No.	Feature	Description
1	Packet length	Total length of one packet.
2	Packet length with direction	Use negative value for uplink-packet and positive value for downlink-packet.
3	Payload length	The length of the dropped payload.
4	Protocol	TLS/SSL, HTTP, TCP, UDP.
5	Burst ID	Number the session streams by burst.
6	Cumulative number of packets	Used to count the cumulative number of packets in each burst.
7	Cumulative bytes of packets	Used to count the cumulative bytes of packets in each burst.
8	Ratio of the number of packets in adjacent bursts	Used to indicate the change in the cumulative number of packets in a burst.
9	Ratio of the total number of bytes in adjacent bursts	Used to indicate the change in the cumulative bytes of packets in a burst.

Nodes. Taking the network flow in Fig. 2 as an example, Fig. 3 shows the corresponding generated MTIG. The nodes in the MTIG represent the packets in the flow. In order to distinguish shorter encrypted malicious flows, each node is associated with information such as the length of the packet, direction of the packet, BURST information and so on. The features embedded into the nodes and their detailed description are listed in Table III.

Edges. All packets in the flow will divide into groups according to burst (defined in Section III-B). For example, a traffic flow is formed as $(P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_{10}, P_{11})$. In this flow, the packets in a burst will be regarded as a group. Consequently, the traffic flow is divided into several small groups as $((P_1), (P_2), (P_3, P_4), (P_5, P_6, P_7), (P_8, P_9), (P_{10}, P_{11}))$. In the MTIG, inside a small group, undirected edges are added to adjacent nodes; between groups, the head and tail nodes of group i are connected to the head and tail nodes of group $i + 1$, respectively. If there is only one node in the group, the node acts as both the head and the tail node.

To explain the MTIG construction process more clearly, the pseudo code of the construction algorithm is shown in Algorithm 1. MTIG takes raw traffic flow as its input. First, a data packet interaction sequence S is created according to the uplink and downlink packets of the flow (line 1). Second, construct groups according to the burst information and initialize the node set V and edge set E (lines 2–3). Then, generate node set according to groups information and embed nine-dimensional features into nodes to enhance graph representation (lines 4–6). After that, add intra-group edges (lines 7–10) and inter-group edges (lines 11–16) according to the conditional loop. Finally, the desired MTIG is generated.

The explanation below shows why MTIG is a powerful representation of multi-dimensional traffic characteristics. First, the nodes of the same layer in MTIG indicate the packets that constitute a burst, which represent the burst-level information of different malicious traffic. Second, the intrinsic characteristics of the flow (e.g., packet length, packet direction, protocol, etc.) are embedded in the graph nodes. The embedding of these features can enhance the representation of shorter malicious flows. Lastly, since the order of packets from the starting of malicious flow to the end of data transmission

Algorithm 1 Construction of Malicious Traffic Interaction Graph**Input:** A bidirectional traffic flow F /* (.pcap) */**Output:** The corresponding MTIG G

```

1: Generate packet interaction sequence  $S$  from flow  $F$ ,  $S = (P_1, \dots, P_N)$ 
2: Separate  $S$  into groups  $B = (g_1, \dots, g_K)$  according to burst information
3: Initialize  $V, E$  as empty sets
4: for  $g_i \in B$  do
5:   for  $P_j \in g_i$  do
6:     Add a node with nine features of  $P_j$  to  $V$ 
7:   for  $g_i \in B$  do
8:     if  $\text{len}(g_i) > 1$  then
9:       for  $v_j \in g_i$  do
10:        Add an edge between  $v_j$  and  $v_{j+1}$  to  $E$ 
11:   for  $g_i \in B$  do
12:     if  $\text{len}(g_i) = 1$  and  $\text{len}(g_{i+1}) = 1$  then
13:       Add an edge between  $g_i$  and  $g_{i+1}$  to  $E$ 
14:     else
15:       Add an edge between the head node of  $g_i$  and the head node of  $g_{i+1}$  to  $E$ 
16:       Add an edge between the tail node of  $g_i$  and the tail node of  $g_{i+1}$  to  $E$ 
17: return  $G$ 

```

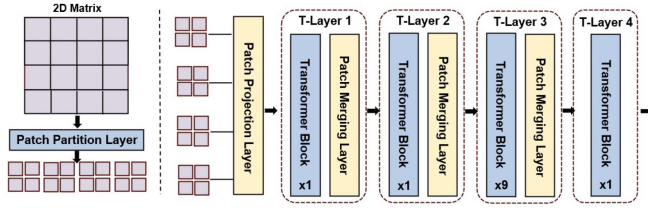


Fig. 6. Transformer module detail diagram.

can be represented by MTIG, the interaction process between client and server is naturally included. In summary, MTIG can contains features of packet burst information, intrinsic features of flows, and network interaction information of flows, each of them has been certified to be valuable for traffic classification [6].

D. Byte Feature Extraction Module

Our approach uses the Transformer architecture that can be computed in parallel rather than CNN or RNN architecture to extract byte-level features. This is because self-attention in the Transformer calculates the correlation between inputs. In malicious traffic classification, self-attention can establish correlations for different parts of the input, so that the information of previous network packets is associated with that of subsequent network packets, and this relationship is considered when doing feature extraction.

The overview of our byte features extraction module (i.e., Transformer module) is presented in Fig. 6. It first partitions the input 2D data matrix into non-overlapping patches by a patch partition layer. Fig. 6 Shows an example when the patch size is set to 2×2 . Through a patch projection layer,

each patch will be flattened to a 1-dimensional vector, which is treated as a token. And each token will be mapped to specific dimension for embedding, as the Transformer Block receives the 1D sequence of token embeddings as input. After that, several Transformer blocks and patch merging layers are applied on these patch tokens. Fig. 6 illustrates the Transformer Block consists of the shifted window-based multi-head self-attention [21], and the two-layer MLP with GELU nonlinearity [36] in between. A LayerNorm (LN) layer [37] is adopted before each multi-head self-attention and each MLP, and each component is connected through the residual structure. The main function of the patch merging layer is to down-sampling the input tokens as the network gets deeper. After the treatment of the patch merging, the length and width of the input feature map is reduced to half of the original, and the depth is doubled.

In this work, several successive Transformer blocks and together with a patch merging layer are called “T-Layer” and set to four. In particular, the fourth T-layer serves as the final layer for feature extraction without the patch merging layer. Each T-Layer uses one Transformer Block [21], except for the third T-Layer, which uses nine Transformer Blocks. These T-Layers jointly produce a hierarchical representation, which has the same feature map resolutions like those of typical convolutional networks (e.g., VGG [22], EfficientNet [23]).

E. Graph Feature Extraction Module

For each flow, the Algorithm 1 in Section III-C is used to generate the corresponding MTIG. The graph neural network become the natural tool of choice, as they can extract features from input graphs automatically. The proposed Graph Feature Extraction module (i.e., GNN module) is composed of two graph attention network (GAT) blocks, two GraphSAGE blocks and a readout layer, as shown in Fig. 1. GAT uses the attention mechanism to distinguish the importance of different adjacent nodes, so that it can better capture the local information and structural information of the graph. However, GAT may face challenges in computing resources when processing large-scale graphs. GraphSAGE can reduce computational overhead by randomly sampling neighbor nodes, thereby processing large-scale graphs more efficiently. By combining GAT with GraphSAGE, we can not only distinguish the importance of different nodes, but also reduce computing and storage overhead while considering global graph features and local node information, making the model more adaptable to graph data of different sizes and densities.

1) *GAT Block*: When we determine whether a graph is a malicious traffic sample, the entire graph structure is considered. We assume that in malicious traffic samples, different nodes contribute differently to the entire graph structure, and therefore, we introduce the graph attention network (GAT [24]). GAT employs the attention mechanism to learn the relative weights between two connecting nodes. The graph convolution operation of GAT is defined as:

$$h_v^{(k)} = \sigma \left(\sum_{u \in N(v) \cup v} \alpha_{vu}^{(k)} W^{(k)} h_u^{(k-1)} \right) \quad (10)$$

where $h_v^{(0)} = X_v \cdot \alpha_{vu}^{(k)}$ is the attention weight, and it measures the connection strength of node v and its neighbor u :

$$\alpha_{vu}^{(k)} = \text{softmax} \left(g \left(a^T \left[W^{(k)} h_v^{(k-1)} \parallel W^{(k)} h_u^{(k-1)} \right] \right) \right) \quad (11)$$

where $g(\cdot)$ is the LeakyReLU activation function and a represents a vector of learnable parameters. The softmax function ensures that the attention weights add up to one in all neighbors of node v . More specifically, we use three GAT layers in one GAT Block. The first GAT layer and the second one contains six-head attention and the third GAT layer uses one-head attention.

2) *GraphSAGE Block*: GCN [25] usually requires saving the entire graph and intermediate update data of all nodes into computer memory. As the graph scale increases, the algorithm consumes more computing and storage resources. Moreover, since the number of a node's neighbors can range from one to a hundred or even more, it is not efficient to obtain all neighbors of the node. To solve this problem, GraphSAGE [26] proposed a new node update method, which limited the number of neighboring nodes participating in the calculation by randomly sampling the neighbors of each node instead of performing calculations on all nodes. It conducts graph convolution by

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot f_k \left(h_v^{(k-1)}, \left\{ h_u^{(k-1)} \forall u \in S_{N(v)} \right\} \right) \right) \quad (12)$$

where $h_v^{(0)} = x_v$, $f_k(\cdot)$ is the aggregation function, and $S_{N(v)}$ represents a random sample of the node v 's neighbors. The aggregation function should be invariant to the permutations of node orderings, such as a mean, sum, min or max function. Specifically, we use three GraphSAGE layers in one GraphSAGE Block, which is the same setup as in GAT Block.

3) *Readout Layer*: After sufficient feature aggregation and updating, the rich information in the graph needs to be summarized to generate smaller representation. The operation of readout is very similar with the operation of pooling, which reduce the size of parameters by down-sampling the data. Through the readout, a compact representation on the graph-level of each graph is obtained, which can be used to predict the malicious traffic label for an entire graph. Common readout operations include summation, average, maximum or minimum. The readout layer is generally defined as

$$h_G = R \left(h_1^{(k)}, h_2^{(k)}, \dots, h_n^{(k)} \right) \quad (13)$$

where n represents the number of nodes in the graph.

F. Classification Module

After byte feature extraction, we get a byte feature vector containing compact byte information, which has dimension D . At the same time, MTIG obtains the graph representation feature through graph feature extraction, which has the same dimension as the byte feature vector. The classification module concatenates the two feature vectors together to obtain a fused traffic representation. Then, it passes through a fully connected layer consisting of two linear layers. After that, softmax maps the fused features to a vector space that is non-negative and sums to 1 to get the final prediction.

V. EXPERIMENTAL SETUP

A. Datasets

To comprehensively evaluate the performance of the proposed method, two datasets are used to conduct experiments, both of which are representative and publicly accessible for encrypted malicious traffic analysis. The Malware Capture Facility Project (MCFP) dataset includes dozens of malware traffic, such as Yakes, HTBot, etc. The USTC-TFC includes twenty categories of malicious and benign traffic.

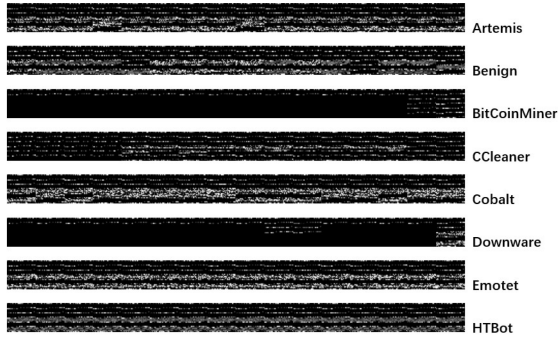
1) The MCFP dataset [30] is collected by the Stratosphere Lab's Malware Capture Facility Project, which continuously obtains both normal and malware traffic captures, and in particular includes the well-known CTU-13 [33]. As MCFP is a normal and malware traffic repository, we utilized the subset of it. We selected nineteen different malware traffic types such as Artemis, BitCoinMiner, HTBot, and Ramit etc. And we selected three normal PCAP files (e.g., "CTU-Normal-30", "CTU-Normal-31" and "CTU-Normal-32") to construct benign traffic category. A total of 284,868 benign and malicious flows were used. The unique identifier (botnet-id) in the MCFP dataset for each class of traffic used (including malware traffic and normal software traffic) is shown in Table IV.

2) The USTC-TFC dataset [31] provides original PCAP files of ten types of malicious traffic (e.g., Geodo, Neris, Nsis-ay, etc.) and ten types of benign traffic (e.g., Gmail, Outlook, Skype, etc.). It has a total size of 3.71 GB with 20 PCAP files, and all PCAP files are separated by traffic category. This dataset is commonly used in encrypted malicious traffic classification [11], [12], [38]. Statistics of the USTC-TFC dataset can be referred to Table V.

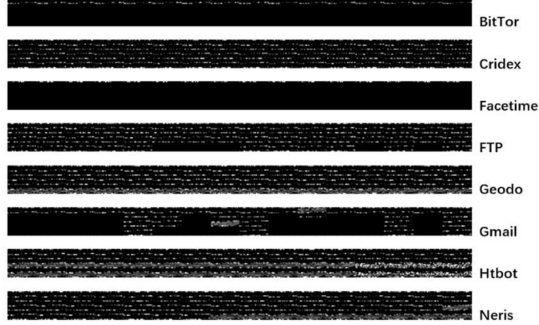
Fig. 7 illustrates the visualization of different classes of malicious traffic. It stretches the first N packets of the session flow and the first L bytes of each packet into a $1 \times M$ ($M=N \times L$) dimensional array. Then convert each position in the byte array to an integer between (0, 255), and finally reshape the byte array into $x \times x$ ($x \leq \lfloor \sqrt{M} \rfloor$) for visualization. As can be seen in Fig. 7, malicious flows such as BitCoinMiner, Downware, and BitTor are typically shorter and contain more zero padding in the tail. The visualization of the malicious flow CCleaner has a discrete distribution of white dots, which is significantly different from normal Benign traffic. Although both the malicious flow Cridex and the malicious flow Geodo show a dotted distribution, Geodo's white dots are more densely packed and have a gray band at the lower edge, which is significantly different from Cridex. These differences can be captured by MTSecurity and used to achieve accurate classification.

B. Evaluation Metrics

All experiments were performed on the platform equipped with an Intel Core 2.7GHz and 16Gb of memory. Pytorch was used as the back-end deep learning framework, and the deep graph library (DGL) was used to construct the malicious traffic interaction graph and design the graph feature extraction module.



(a) MCFP dataset



(b) USTC-TFC dataset

Fig. 7. Visualization of different classes of malicious traffic. We selected eight classes for visualization on both datasets. (a) is on the MCFP dataset, (b) is on the USTC-TFC dataset.

To evaluate and compare the performance of the proposed model, we used several typical metrics, including Accuracy (ACC), Precision (PR), Recall (RC), F1-score, and False Positive Rate (FPR). The formulas are shown in (14)–(18). The Macro Average specifically refers to calculating the mean value of each category, which is used to avoid biased results due to data imbalance. This formula is defined in (19). Where n represents the total number of categories, and X represents a certain evaluation metric.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (14)$$

$$Precision = \frac{TP}{TP + FP} \quad (15)$$

$$Recall = \frac{TP}{TP + FN} \quad (16)$$

$$FPR = \frac{FP}{TN + FP} \quad (17)$$

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (18)$$

$$Macro\ Average - X = \frac{1}{n} \sum_{i=1}^n X_i \quad (19)$$

C. Data Preprocessing

We perform supervised training and testing on the MCFP [30] and USTC-TFC [31] datasets. Prior to the experiments, we use the SplitCap tool to split the raw traffic into separate bi-directional flows according to quintuples. Then we label each flow with the corresponding traffic category as

TABLE IV
NUMBER OF BI-FLOW SAMPLES FOR DIFFERENT LABELS AND BOTNET-ID IN THE MCFP DATASET

Label	Flows	Botnet-ID
Benign	51530	Normal-30, Normal-31, Normal-32
Adload	214	349-1
Artemis	10252	316-1
BitCoinMiner	1023	347-1
CCleaner	7614	320-1,320-2
Cobalt	250	345-1
Downware	1039	336-1
Dridex	10029	346-1
Emotet	24730	276-2
HTBot	36852	348-1
MagicHound	30431	318-1
MinerTrojan	1800	342-1
PUA	2648	213-1
Ramnit	456	343-1
Tinba	3195	225-1
Trickbot	30052	327-1
Trickster	54607	323-1
TrojanDownloader	1016	341-1
Ursnif	10558	313-1
WebCompanion	6572	339-1

TABLE V
NUMBER OF BI-FLOW SAMPLES FOR DIFFERENT LABELS IN THE USTC-TFC DATASET

Label	Flows	Label	Flows
BitTor	4491	Nsis-ay	1862
Cridex	8189	Outlook	7370
Facetime	6000	Shifu	9631
FTP	10000	Skype	5679
Geodo	6033	SMB	10000
Gmail	5252	Tinba	7749
Htbot	4298	Virut	5788
Miuref	4900	Weibo	10000
MySQL	9992	Wow	7786
Neris	6511	Zeus	10000

described in the paper. For the MCFP dataset, we parse the SSL/TLS protocol and the HTTP protocol. For the USTC-TFC dataset, we parse the SSL/TLS and HTTP protocols, as well as other data transfer protocols such as DNS, MySQL, and FTP. All flows are packet parsed and anonymized as described in Section IV-B and raw traffic representations are generated. The corresponding MTIGs are generated according to the graph construction algorithm described in Section IV-C. Since the extreme imbalance of the original data makes it impossible to train the model effectively on the categories with less data, with reference to [11], we use up-sampling [35] to increase the sample size, and random down-sampling [35] to reduce the number of samples in some categories to make the input data relatively balanced. Statistics of the MCFP and the USTC-TFC datasets are listed in Table IV and Table V. Each dataset used in the experiments was divided into no-overlapping training set, validation set and testing set following the ratio of 6 : 2 : 2.

D. Hyperparameter Selection

In this subsection, we describe the hyperparameters used in MTSecurity. We use the MCFP dataset [30] and the USTC-TFC dataset [31] for model parameter optimization. The hyperparameters are tuned to adjust the trade-off between variance, bias and classification performance. In order to get better training results, we perform hyperparameter selection for the Transformer module and GNN module separately.

Due to the large number of training examples and hyperparameters in MTSecurity, it is a challenging task to find the best hyperparameter settings. Therefore, for the Transformer module, we refer to the article [21] to set the appropriate hyperparameters and make fine-tuning of the results. For the GNN module, we refer to [13] to search hyperparameters from a listed area and choose the best values. Specifically, for GNN activation function, the search area is [Tanh, Relu, Elu], the candidate parameter area of GNN hidden units is [16, 32, 64, 128] the candidate parameter area of GNN dropout rate is [0.1, 0.2, 0.3], and the search area of GNN readout type is [Mean, Sum]. For each GAT block, where the number of GAT layers is selected between [1], [3], [6], the GraphSAGE block follows the same principle. In order to seek an efficient match between the GAT block and the GraphSAGE block, we also compared the results using different GAT/GraphSAGE. It is worth noting that the number of heads in attention can split the embedded input data into different heads to avoid losing local information in a large matrix. That is to say using different number of multiple heads also affects the performance of graph feature extraction. Therefore, we also tested the effect of different number of heads on the performance of the GNN module. The experimental results for GAT/GraphSAGE as well as for the number of heads are shown in Table VI and Table VII (training for 30 epochs). In above tables, GAT/GraphSAGE denotes the ratio of the number of GAT blocks to the number of GraphSAGE blocks. GAT layer denotes the number of GAT layers in each GAT block. GraphSAGE layer denotes the number of GraphSAGE layers in each GraphSAGE block. Heads denotes the number of self-attention heads in each GAT layer. We use accuracy and F1 as metrics for evaluating performance, and they are defined as shown in Section V-B.

As for the selection of N and L (described in Section IV-B), we refer to the article [19] and use the optimal results therein $N = 7$ and $L = 125$, thus obtaining $M = 875$. Since the patch size in the Transformer module is 4 [21], we truncate M to 784 and obtain the input dimension of the Transformer module to be 28×28 . The reason for this is that 28 can be divided by 4 so that the input data can be processed without padding. The final values chosen for the hyperparameters are shown in Table VIII. Based on the parameters in the table we get the results given in the experiment section.

VI. EXPERIMENT RESULTS AND DISCUSSION

A. Graph Node Quantity Selection

The number of graph nodes used to construct the corresponding MTIG also has an impact on the performance of MTSecurity, where one graph node in each MTIG represents

TABLE VI
GNN HYPERPARAMETER SELECTION ON MCFP DATASET

Proportions	GAT Layers	GraphSAGE Layers	Heads	ACC	F1
GAT/GraphSAGE = 1	1	1	1	0.7565	0.4739
			3	0.7830	0.5928
			6	0.7854	0.5862
	3	3	1	0.7898	0.6002
			3	0.8038	0.6013
			6	0.8051	0.5925
	6	6	1	0.7811	0.5706
			3	0.8030	0.5953
			6	0.8057	0.6069
GAT/GraphSAGE = 2	1	1	1	0.7771	0.5705
			3	0.8002	0.5717
			6	0.8009	0.6101
	3	3	1	0.7871	0.5914
			3	0.8030	0.5763
			6	0.8068	0.6140
	6	6	1	0.7648	0.5480
			3	0.7972	0.5912
			6	0.8002	0.6113

TABLE VII
GNN HYPERPARAMETER SELECTION ON USTC-TFC DATASET

Proportions	GAT Layers	GraphSAGE Layers	Heads	ACC	F1
GAT/GraphSAGE = 1	1	1	1	0.6942	0.6177
			3	0.6795	0.6147
			6	0.6671	0.5978
	3	3	1	0.7169	0.6693
			3	0.7391	0.7069
			6	0.7815	0.7464
	6	6	1	0.7374	0.6621
			3	0.7680	0.7239
			6	0.7351	0.6857
GAT/GraphSAGE = 2	1	1	1	0.6822	0.6173
			3	0.7351	0.7071
			6	0.7465	0.7153
	3	3	1	0.7296	0.6871
			3	0.7750	0.7233
			6	0.7631	0.7241
	6	6	1	0.4425	0.3597
			3	0.6679	0.6038
			6	0.7687	0.7152

TABLE VIII
HYPERPARAMETER SELECTION FOR MTSECURITY

Hyperparameter	Values
Transformer Input Size	28 ²
Transformer Patch size	4
Transformer Embedding Dimension	128
GNN Activation Function	Elu
GNN Hidden Units	64
GNN Dropout	0.2
GNN Readout Type	Mean
Number of Layer in one GAT Block	3
Number of Layer in one GraphSAGE Block	3
GAT/GraphSAGE	2*, 1+
Training Epochs	100
Batch Size	32
Learning Rate	0.0001
Optimizer	Adam
Loss Function	CrossEntropyLoss

Notes: * is MCFP Datasets, + is USTC-TFC Datasets

one packet in each flow. Since the number of packets in a flow varies from a few to hundreds of thousands, in order to select the appropriate number of graph nodes, we counted the distribution of the number of packets for all flows in the

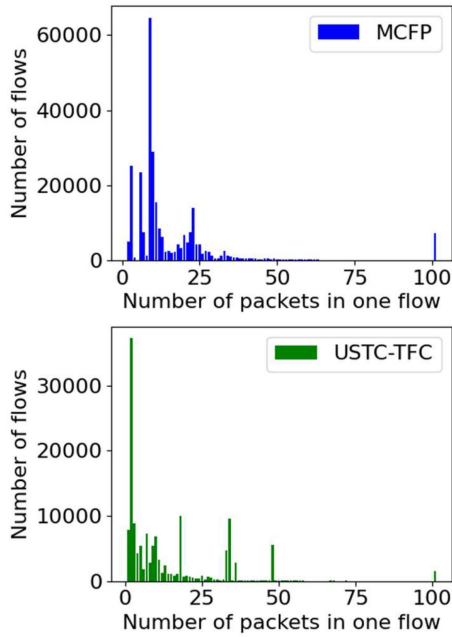


Fig. 8. Distribution of the number of packets in the MCFP dataset (blue) and USTC-TFC dataset (green). Since the number of packets in a flow can reach hundreds of thousands and cannot be fully displayed in the figure, flows with a number of packets greater than 100 are truncated to 100 for visualization.

MCFP dataset and the USTC-TFC dataset, and the results are shown in Fig. 8. The results show that more than 90% of the flows have packet counts between 0 and 50. In the MCFP dataset, flows with packet counts greater than or equal to 100 account for only 2.59% of the total flows, and in the USTC-TFC dataset, this percentage is 1.08%. Although the higher the number of packets used to construct the graph, the more distinct the discriminative features of the graph representation. However, as the number of graph nodes used increases, the time to construct the graph and the time to perform graph representation learning also increases [13]. Therefore, we select the first 50 packets of each stream to construct the graph to balance the performance and time consumption.

B. Comparison of Graph Construction Methods

To evaluate the graph representation capability of the constructed MTIG, we use the same GNN classifier to extract features and classify MTIG and TIG [13] respectively. We train and test on MCFP dataset and USTC-TFC dataset respectively to comprehensively evaluate the effectiveness of the constructed graph representation method.

For efficient graph feature extraction, we use the GNN module proposed in this paper for graph feature extraction. The graph feature extraction module connects a fully connected layer and a softmax layer to form a complete graph classifier. Note that the same graph classifier and the same training parameters are used for both comparison groups in order to avoid performance differences due to model structure and parameters.

Result on MCFP: As shown in Fig. 9 (a), we can see that the accuracy of our graph representation method improves very quickly, reaching 69.25% in the second epoch, while the

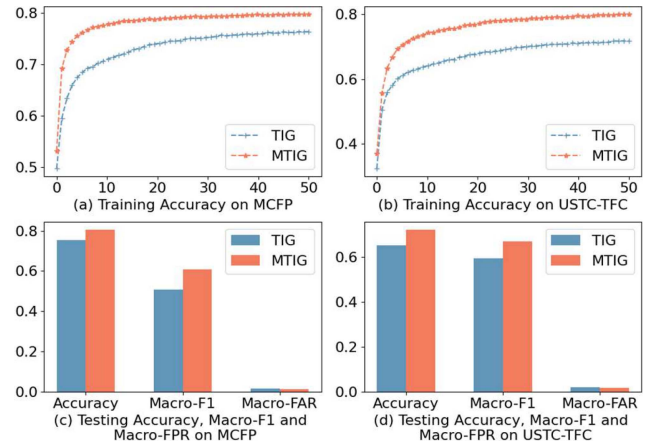


Fig. 9. Compare TIG and MTIG using the same graph classifier on the MCFP dataset and USTC-TFC dataset, respectively. (a) Training Accuracy on MCFP dataset. (b) Training Accuracy on USTC-TFC dataset. (c) Testing Accuracy, Macro-F1 and Macro-FPR on MCFP dataset. (d) Testing Accuracy, Macro-F1 and Macro-FPR on USTC-TFC dataset.

accuracy of TIG is only 59.55%. And MTIG consistently maintains better accuracy during the training process. After training for 50 epochs on the training set, we conduct further evaluation on the testing set, as shown in Fig. 9 (c). Compared to TIG, the new graph representation method got 6.86% increase on accuracy, 19.74% improvement on macro-F1 score and 21.68% decrease on FPR.

Result on USTC-TFC: As shown in Fig. 9 (b), we can see that MTIG has also consistently maintained higher accuracy during training on the USTC-TFC dataset. After training for 50 epochs on the training set, we conduct further evaluation on the testing set, as shown in Fig. 9 (d). MTIG outperforms the TIG in all indicators. Compared to TIG, the new graph representation method got 9.72% increase on accuracy, 11.33% improvement on macro-F1 score and 20.11% decrease on FPR.

The following analysis explains why MTIG is better than TIG.

TIG has been introduced for processing decentralized encrypted traffic [13]. Since such flows are typically long, the resulting TIGs are highly distinguishable. However, the scenario differs when confronted with malicious flows. Some malware-generated malicious flows are notably brief, comprising only 1 to a few packets. If framed in the same manner as TIG, the distinguishability between different malicious flows diminishes. To address this challenge, we propose a new method called MTIG. In comparison to TIG, MTIG incorporates nine additional dimensions of discriminative features into the graph nodes to enhance the representation characteristics of malicious traffic (as illustrated in Table III). These added statistical features ensure that MTIG remains highly distinguishable even when dealing with shorter flows. As a result, MTIG holds significant advantages in recognizing malicious encrypted traffic compared to TIG.

C. Experiment Results

In our experiments, we mainly focus on the multi-class classification task for malware traffic classification. To evaluate the capability of the proposed method, we first train and

TABLE IX
RESULTS ON THE MCFP DATASET

Method	ACC	PR	RC	F1
GraphDApp [13]	0.4386	0.1607	0.2297	0.1675
Single RF [28]	-	-	-	0.6360
MTHL [27]	-	-	-	0.6390
Multi-RF [28]	-	-	-	0.6770
ET-Bert [12]	0.7451	0.7672	0.7147	0.7073
R1DiT [19]	-	-	-	0.9750
MTSecurity(ours)	0.9946	0.9954	0.9926	0.9940

TABLE X
RESULTS ON THE USTC-TFC DATASET

Method	ACC	PR	RC	F1
GraphDApp [13]	0.8789	0.8226	0.8260	0.8234
AppScanner [5]	0.8954	0.8984	0.8968	0.8892
FS-Net [29]	0.8846	0.8846	0.8920	0.8840
Deeppacket [10]	0.9640	0.9650	0.9631	0.9641
TSCRNN [11]	-	0.9870	0.9860	0.9870
ET-Bert [12]	0.9741	0.9738	0.9730	0.9719
MTSecurity(ours)	0.9948	0.9937	0.9931	0.9934

test our model on the MCFP dataset. After that, to show the generality of the proposed method on other network traffic sources, we further validated our model on USTC-TFC dataset. On the MCFP dataset, we selected five different methods (GraphDApp [13], Single-RF [28], MTHL [27], Multi-RF [28], ET-Bert [12] and R1DiT [19]). On the USTC-TFC dataset, we compared it with six advanced methods (AppScanner [5], FS-Net [29], Deeppacket [10], GraphDApp [13], TSCRNN [11] and ET-Bert [12]).

As shown in Table IX and Table X, these methods exhibit large performance differences in accuracy, precision, recall and F1-score. It is clear that the classification performance of single feature input methods is lower than that of fused feature input methods.

Result on MCFP: MTSecurity achieves the best performance on accuracy, precision, recall and F1-score as 0.9946, 0.9954, 0.9926 and 0.9940. Our proposed model has improved by 1.90% on F1-score compared to the best existing model [19]. Compared with the Graph-based method [13], our proposed method has improved accuracy by more than 50% and F1 score by more than 80%.

Result on USTC-TFC: MTSecurity achieves accuracy, precision, recall and F1-score as 0.9948, 0.9937, 0.9931 and 0.9934. Our proposed method outperforms the latest ET-Bert method by 2.07% on accuracy and 2.15% on F1-score. At the same time, MTSecurity outperforms the Graph-based method by 11.59% in accuracy and 17.00% in F1-score.

This phenomenon suggests that graphs, as an auxiliary feature, can significantly improve the upper bound of the accuracy of the raw data-based approach to more than 0.99. In conclusion, it can be seen that the proposed method is the most accurate among these methods and has significant improvement over the existing models.

In order to evaluate the proposed method more comprehensively, we set up two other control groups. The first one

TABLE XI
ACCURACY, PRECISION, RECALL AND MACRO F1-SCORE OF TWO CONTROL GROUP AND THE PROPOSED MODEL ON MCFP DATASET

Method	Graph (GNN)	Raw (Transformer)	MTSecurity (ours)
ACC	0.8078	0.9834	0.9946
PR	0.6843	0.9780	0.9954
RC	0.6618	0.9826	0.9926
F1	0.6231	0.9798	0.9940
FPR	0.0110	0.0010	0.0003

TABLE XII
ACCURACY, PRECISION, RECALL AND MACRO F1-SCORE OF TWO CONTROL GROUP AND THE PROPOSED MODEL ON USTC-TFC DATASET

Method	Graph (GNN)	Raw (Transformer)	MTSecurity (ours)
ACC	0.7955	0.9805	0.9948
PR	0.7572	0.9736	0.9937
RC	0.7831	0.9762	0.9931
F1	0.7595	0.9745	0.9934
FPR	0.0103	0.0010	0.0003

is named Graph, which utilizes only MTIG as input. In this control group, only GNN module is available for feature extraction. The second one is named Raw, which uses only raw traffic data as input, and only Transformer module is included for feature extraction. It should be noted that the outputs of the feature extraction module of the two control groups are directly connected to the classification module without feature concatenation. Furthermore, to maintain the uniqueness of the variables, the parameters utilized in both training and testing phases are kept consistent with those employed in MTSecurity.

Table XI lists the various performance metrics of the two control groups and the proposed method on the MCFP dataset. From the table, compared to single-GNN, MTSecurity method got 18.78% increase on accuracy, 37.31% improvement in macro-F1 score and 97.27% decrease in FPR. Meanwhile, compared to single-Transformer, MTSecurity method got 1.13% increase on accuracy, 1.43% improvement in macro-F1 score and 70% decrease in FPR.

Table XII lists the various performance metrics of the two control groups and the proposed method on the USTC-TFC dataset. From the table, compared to single-GNN, MTSecurity method got 20.03% increase on accuracy, 23.55% improvement in macro-F1 score and 97.09% decrease in FPR. Meanwhile, compared to single-Transformer, MTSecurity method got 1.44% increase on accuracy, 1.90% improvement in macro-F1 score and 70% decrease in FPR.

The above experimental results show that both input data are beneficial in providing complementary features for classification. This result also demonstrates the effectiveness of the proposed method based on graph and raw data in malicious traffic detection.

In order to visualize the classification performance of the proposed MTSecurity model, Table XI details the various metrics for each traffic class generated by the proposed method on the MCFP dataset. Table XII details the metrics of the proposed method on the USTC-TFC dataset for each traffic

TABLE XIII
THE DETAILED PRECISION, RECALL, F1-SCORE OF
EACH TRAFFIC CLASS OF MCFP DATASET

Label	PR	RC	F1
Adload	1.0000	1.0000	1.0000
Artemis	0.9995	0.9985	0.9990
Benign	0.9990	0.9984	0.9991
BitcoinMiner	1.0000	0.9853	0.9926
CCleaner	0.9973	0.9777	0.9874
Cobalt	1.0000	1.0000	1.0000
Downware	0.9951	0.9855	0.9903
Dridex	0.9995	0.9985	0.9990
Emotet	0.9941	0.9939	0.9940
HTBot	0.9710	0.9255	0.9477
MagicHound	0.9890	0.9651	0.9769
MinerTrojan	1.0000	0.9993	0.9996
PUA	1.0000	1.0000	1.0000
Ramnit	0.9990	0.9990	0.9990
Tinba	0.9991	0.9974	0.9982
Trickbot	1.0000	0.9994	0.9997
Trickster	0.9185	0.9732	0.9451
TrojanDownloader	0.9965	1.0000	0.9982
Ursnif	1.0000	1.0000	1.0000
WebCompanion	0.9995	1.0000	0.9997

TABLE XIV
THE DETAILED PRECISION, RECALL, F1-SCORE OF
EACH TRAFFIC CLASS OF USTC-TFC DATASET

Label	PR	RC	F1
BitTor	1.0000	0.9989	0.9994
Cridex	1.0000	1.0000	1.0000
Facetime	0.9995	1.0000	0.9997
FTP	0.9983	0.9992	0.9987
Geodo	1.0000	0.9992	0.9996
Gmail	0.9990	0.9990	0.9990
Htbot	0.9965	1.0000	0.9982
Miuref	1.0000	1.0000	1.0000
MySQL	1.0000	1.0000	1.0000
Neris	0.9710	0.9255	0.9477
Nsis-ay	0.9890	0.9651	0.9769
Outlook	1.0000	0.9993	0.9996
Shifu	1.0000	1.0000	1.0000
Skype	0.9990	0.9990	0.9990
SMB	0.9991	0.9974	0.9982
Tinba	1.0000	0.9994	0.9997
Virus	0.9185	0.9732	0.9451
Weibo	0.9965	1.0000	0.9982
Wow	1.0000	1.0000	1.0000
Zeus	0.9995	1.0000	0.9997

class. As can be seen from Table XIII and Table XIV, the proposed method performs well in all classes of malicious traffic.

VII. THREATS TO VALIDITY

The potential threat of our study is that differences in the experimental environment may affect the accuracy of the results. To eliminate this threat, we strictly controlled the before and after parameters to be consistent in the comparison experiments to avoid the potential threat caused by inconsistent implementation. At the same time, the same experiments were conducted on multiple platforms and devices to ensure

the consistency and reproducibility of the results. Another potential threat is that our experiments were conducted in a relatively controlled environment, and external validity threats cannot be avoided, i.e., there may be some limitations and validity challenges in generalizing the experimental results to a wider range of scenarios. In the follow-up study, we plan to expand the malicious encrypted traffic dataset by introducing more diverse types of malicious traffic. Without training these new samples, we will evaluate the ability of the proposed algorithm to classify these unknown traffic as benign/malicious (i.e., binary classification), so as to comprehensively test and evaluate the effectiveness and generalization ability of the proposed method.

VIII. CONCLUSION

Encrypted malicious traffic classification plays a crucial role in identifying and preventing malicious activities, thereby enhancing network security. This paper explores existing approaches to encrypted malicious traffic classification and introduces a novel method, MTSecurity, leveraging Transformer and graph neural networks. MTSecurity autonomously detects encrypted malicious traffic by combining the original byte features of encrypted malicious flows with graph-based traffic interaction features. The fusion of the two features can describe the behavior of encrypted malicious flows more comprehensively than any individual feature. Numerous experimental results validate the effectiveness of MTSecurity. Through the fusion of byte features and network interaction features, MTSecurity achieves a classification accuracy of 99.48%, surpassing analyses based on single features. Additionally, experimental findings indicate that the proposed method significantly enhances the classification accuracy of unbalanced, invisible, and multi-category malicious traffic compared to state-of-the-art methods. However, the introduction of two classifiers raises concerns about slow inference speed, and further exploration is needed to better integrate these two features. In the future, MTSecurity will undergo further optimization to enhance its inference speed and performance.

ACKNOWLEDGMENT

The authors want to convey our grateful appreciation to the corresponding author of this paper, Yulin Lei. He has offered advice with huge values in all stages when writing this essay to us. The authors would like to express their sincere appreciation to the editors and the anonymous reviewers for their insightful comments, which have greatly helped us improve the quality of this article.

REFERENCES

- [1] "HTTPS encryption on the Web." 2019. [Online]. Available: <https://transparencyreport.google.com/https/overview>
- [2] "Security outcomes report, volume 3." 2019. [Online]. Available: https://www.cisco.com/c/dam/global/zh_cn/products/security/security-reports/threats-of-the-year-cybersecurity-series.pdf
- [3] "The state of encrypted attacks." 2021. [Online]. Available: <https://www.zscaler.com/resources/infographics/infographic-state-of-encrypted-attacks-2021.pdf>

- [4] A. S. Shekhawat, F. Di Troia, and M. Stamp, "Feature analysis of encrypted malicious traffic," *Expert Syst. Appl.*, vol. 125, pp. 130–141, Jul. 2019.
- [5] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, pp. 63–78, 2017.
- [6] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, "Fine-grained Webpage fingerprinting using only packet length information of encrypted traffic," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2046–2059, 2020.
- [7] Z. Wang, K. W. Fok, and V. L. L. Thing, "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102542.
- [8] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, 2017, pp. 43–48.
- [9] T. Shapira and Y. Shavitt, "Flowpic: Encrypted Internet traffic classification is as easy as image recognition," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2019, pp. 680–687.
- [10] M. Lotfollahi, M. J. Siavoshani, R. S. Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [11] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [12] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [13] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [14] W. Wang, Y. Shang, Y. He, Y. Li, and J. Liu, "BotMark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors," *Inf. Sci.*, vol. 511, pp. 284–296, Feb. 2020.
- [15] Y. Li et al., "GraphDDoS: Effective DDoS attack detection using graph neural networks," in *Proc. IEEE 25th Int. Conf. Comput. Support. Cooperat. Work Des. (CSCWD)*, 2022, pp. 1275–1280.
- [16] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. 10th Annu. ACM Workshop Privacy Electron. Soc.*, 2011, pp. 103–114.
- [17] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [18] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [19] O. Barut, Y. Luo, P. Li, and T. Zhang, "RIDIT: Privacy-preserving malware traffic classification with attention-based neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 2071–2085, Jun. 2023.
- [20] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022.
- [21] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.* 2021, pp. 10012–10022.
- [22] K. Simonyana and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [23] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.
- [24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [25] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [26] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, 2017, p. 30.
- [27] O. Barut, Y. Luo, T. Zhang, W. Li, and P. Li, "Multi-task hierarchical learning based network traffic analytics," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2021, pp. 1–6.
- [28] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [29] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
- [30] 2015, "Stratosphere Laboratory Datasets," stratosphereips. Dataset, Mar. 2020, <https://www.stratosphereips.org/datasets-overview>.
- [31] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717.
- [32] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Comput. Secur.*, vol. 45, pp. 100–123, Sep. 2014.
- [33] M. Korczyński and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2014, pp. 781–789.
- [34] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order Markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Security*, vol. 12, pp. 1830–1843, 2017.
- [35] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," 2016, *arXiv:1603.07285*.
- [36] Q. Zhang, C. Wang, H. Wu, C. Xin, and T. V. Phuong, "GELU-Net: A globally encrypted, locally unencrypted deep neural network for privacy-preserved learning," in *Proc. IJCAI*, 2018, pp. 3933–3939.
- [37] J. Xu, X. Sun, Z. Zhang, G. Zhao, and J. Lin, "Understanding and improving layer normalization," in *Proc. 33rd Conf. Neural Inf. Process. Syst.*, 2019, pp. 1–11.
- [38] J. Dai, X. Xu, and F. Xiao, "GLADS: A global-local attention data selection model for multimodal multitask encrypted traffic classification of IoT," *Comput. Netw.*, vol. 225, Apr. 2023, Art. no. 109652.
- [39] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, 2017, pp. 1–11.



Jin Yang received the M.S. and Ph.D. degrees in computer science from Sichuan University, Sichuan, China, in 2004 and 2007, respectively, where he is currently an Associate Professor with the School of Cyber Science and Engineering, Sichuan University. His main research interests include network security, knowledge discovery, and expert systems.



Xinyun Jiang is currently pursuing the M.S. degree with the School of Cyber Science and Engineering, Sichuan University, Sichuan, China. Her main research interests include artificial intelligence, network intrusion detection, encrypted traffic classification, and network security.



Yulin Lei is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Sichuan University, Sichuan, China. His main areas of interest and research are intrusion detection, malicious traffic, artificial intelligence, and network security.



Weiheng Liang is currently pursuing the M.S. degree in electronic information with the School of Cyber Science and Engineering, Sichuan University, Sichuan, China. His main research interests include encrypted traffic detection, network security, and deep learning.



Siyu Li received the B.E. degree from Sichuan University, Chengdu, China, in 2021, where he is currently pursuing the M.S. degree with the School of Cyber Science and Engineering. His research interests include natural language processing, Transformer-based models, and social network analysis.



Zicheng Ma received the B.E. degree from the Sichuan University, Chengdu, China, in 2021, where he is currently pursuing the M.S. degree with the School of Cyber Science and Engineering. His research areas are artificial intelligence, natural language processing, and visualization technology.